Incentive-Aware Multi-Objective Matching for Peer-to-Peer Commuter Carpooling

CS6140 Machine Learning · Final Project

Krunal Savaj
Khoury College of Computer Sciences · Northeastern University

April 2026

The average US car carries 1.08 passengers during peak hours [9]

- Uber/Lyft are built for professional drivers, not regular commuters
- Commuters have fixed routes and low detour tolerance
- The core difficulty: the same 1.8 km detour can be acceptable or unacceptable depending on the fare and trip length
- Threshold-based matching systems (e.g., "accept if detour < 3 km") ignore fare and timing interactions and treat all pairs within the threshold identically

> "If 10% of NYC solo commuters shared rides, the city could remove 300,000 cars from morning traffic." (Santi et al., PNAS 2014)

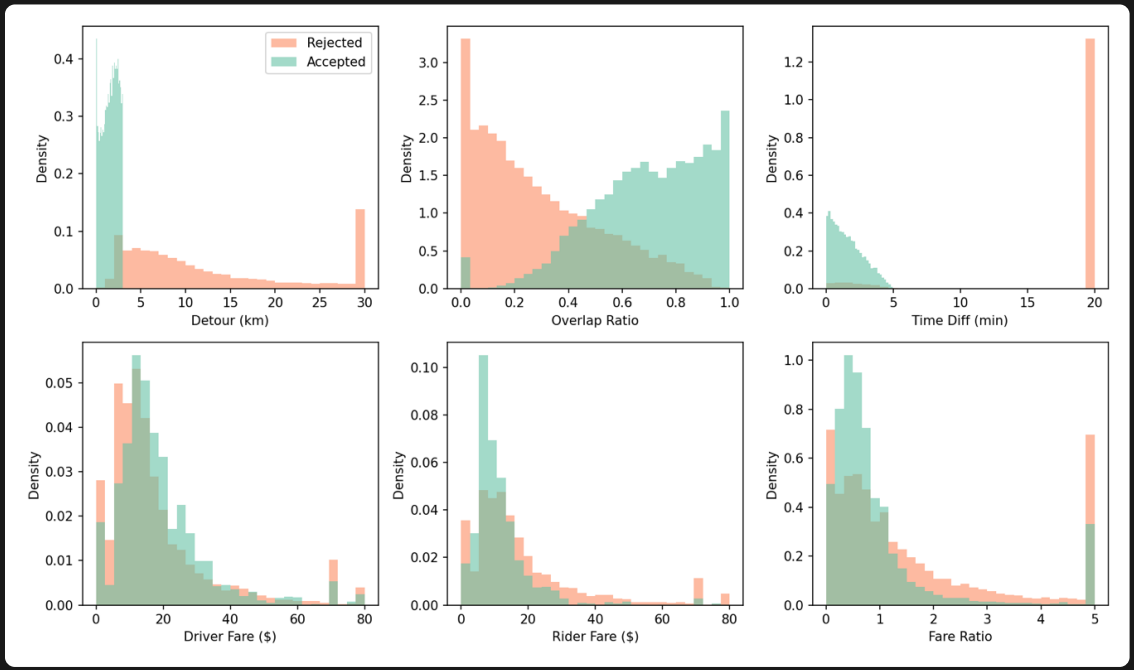
```
Driver: Bronx → Midtown (8 km)
   ↓
+ Rider B: 1.8 km detour, $22 fare  [ACCEPTED]
+ Rider C: 1.8 km detour,  $3 fare  [REJECTED]

Same detour. Different economics.
Only ML can tell the difference.
```

No P2P carpooling dataset exists, so we built one

- **Source:** NYC TLC Yellow Taxi, 3.5M trips, August 2025. Used 25% sample = ~268,000 commute-window trips.
- **Problem:** TLC removed GPS coordinates in 2016 (privacy). Only zone IDs remain — coordinate synthesis required (next slide).
- **Time windows:** AM 7–10 and PM 4–7. These are the only windows where commuter carpooling is realistic.
- **Simulation:** 50% of trips randomly assigned as Drivers, 50% as Riders. For each nearby pair, compute features and assign an acceptance label.
- **Label policy:** 4-factor composite + Gumbel noise to simulate human irrationality:
  - detour severity (30%) + fare ratio (25%) + time gap (25%) + trip ratio (20%)
  - label = 1 if composite + Gumbel noise > 0.45, else 0
- **Train/test split:** driver-level 80/20 — all pairs for a given driver stay in one partition, preventing entity leakage

What the 38,102 training pairs look like



Feature distributions by acceptance label. Green = accepted, pink = rejected.

Dataset summary

Split	Pairs	Share
Accepted (label = 1)	16,623	43.6%
Rejected (label = 0)	21,479	56.4%
Total	**38,102**	**100%**

Key patterns

- **Detour:** accepted pairs cluster 0–5 km; rejected spread to 80 km (genuine negatives beyond threshold)
- **Overlap ratio:** accepted near 0.8–1.0; rejected near 0.0–0.2
- **Time diff:** rejected spike at 15-min cutoff; accepted cluster near 0–5 min
- **Fares:** distributions overlap — fare *ratio* is more informative than raw fare alone

> Near-balanced classes (44%/56%). XGBoost scale_pos_weight = 1.28 handles the minor imbalance.

How we synthesize exact GPS coordinates from zone IDs

1. **Bounding box scatter**
   Sample (lat, lon) uniformly inside the zone's rectangular bounds. Avoids the artificial clustering problem of centroid-only assignment.

2. **Point-in-polygon validation**
   Reject any point outside the actual polygon (coastal zones extend over water). Resample up to 10 rounds.

3. **Corridor-aware bearing**
   Compute true compass direction from pickup zone centroid to dropoff zone centroid. Add Gaussian noise ($\sigma = 0.3$ rad $\approx 17°$) for street-level variation.

4. **Spherical projection**
   Project dropoff exactly trip_distance miles along that bearing using spherical trigonometry.

Bearing formula (forward azimuth)

$$x = \sin(\Delta\lambda) \cdot \cos(\varphi_2)$$
$$y = \cos(\varphi_1) \cdot \sin(\varphi_2) - \sin(\varphi_1) \cdot \cos(\varphi_2) \cdot \cos(\Delta\lambda)$$
$$\theta = \operatorname{atan2}(x, y) + N(0, 0.3)$$

Spherical dropoff projection

$$d/R = \text{trip_distance} / 3958.8$$

$$\text{lat}_2 = \arcsin\left(\sin(\text{lat}_1) \cdot \cos(d/R) + \cos(\text{lat}_1) \cdot \sin(d/R) \cdot \cos(\theta) \right)$$

$$\text{lon}_2 = \text{lon}_1 + \operatorname{atan2}\left(\sin(\theta) \cdot \sin(d/R) \cdot \cos(\text{lat}_1), \cos(d/R) - \sin(\text{lat}_1) \cdot \sin(\text{lat}_2) \right)$$

Result: Haversine distance of synthesized coordinates matches the real taxi meter distance exactly.

Dual-objective XGBoost + bipartite matching

Feature vector (9 features per pair)

```
x = [ detour_km, overlap_ratio, time_diff_mins,
      driver_fare, rider_fare, fare_ratio,
      pickup_hour, pickup_dow, shareability_count ]

overlap_ratio = driver_orig_dist / (driver_orig_dist + detour_km),
clamped [0,1]
shareability_count = riders within 5 km in same time batch [1]
```

Two prediction objectives

Model A (Regressor): $\hat{y}_{det} = f_{reg}(x) \rightarrow$ detour km

Model B (Classifier): $\hat{y}_{acc} = f_{clf}(x) \rightarrow$ P(accept)

Score: $s_{ij} = \hat{y}_{acc} - 0.1 \cdot \hat{y}_{det}$

($\lambda = 0.1$ is a tunable penalty weight; higher λ penalises detour more)

Bipartite optimization at inference

$$\max \sum_{ij} s_{ij} \cdot x_{ij}$$

$$\text{s.t.} \sum_{j} x_{ij} \leq 1 \quad \text{(each driver matched once)}$$

$$\sum_{i} x_{ij} \leq 1 \quad \text{(each rider matched once)}$$

- **Why XGBoost?** Gradient-boosted trees outperform neural networks on tabular geospatial data (Grinsztajn et al., NeurIPS 2022)
- **Why dual-model?** Detour is a regression target; acceptance is a classification target. One model cannot serve both.
- **Why bipartite at inference only?** Training on ALL feasible pairs (not just optimal ones) prevents selection bias: the model learns the full distribution of match quality, not only the "winning" pairs.
- **Theoretical guarantee:** Learning-augmented matching provably outperforms pure heuristics or pure ML alone (Bjorck et al., AAAI 2021)

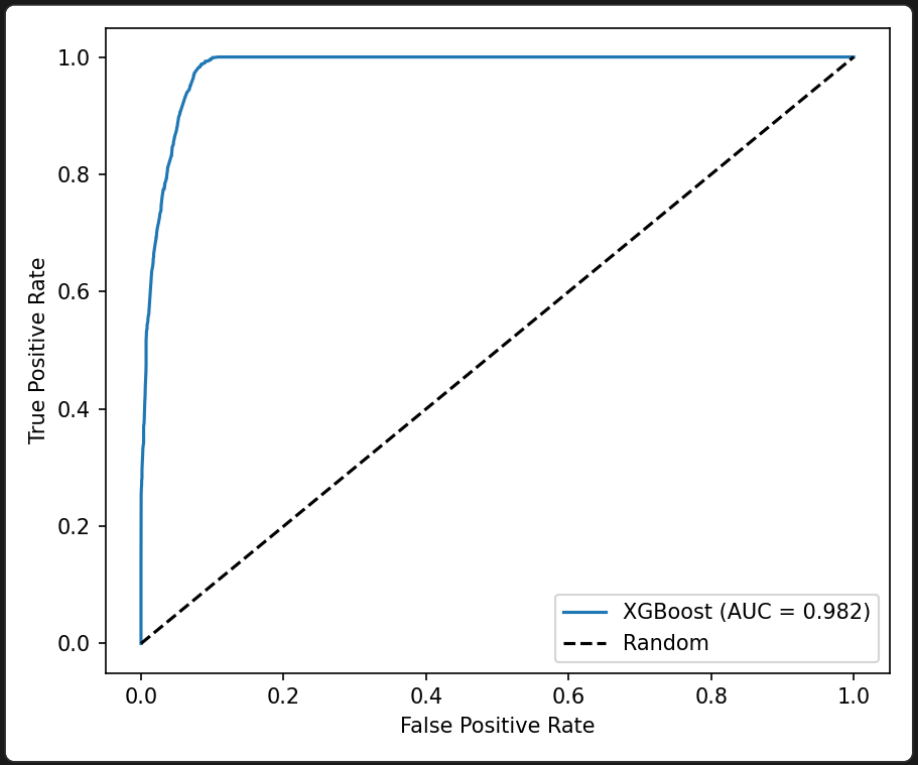
XGBoost produces better rankings, not just higher accuracy

Held-out test set: 7,677 pairs from 3,969 unseen drivers (driver-level 80/20 split)

Model	Accuracy	F1	ROC-AUC
Rule-Based (distance cutoff)	93.3%	92.9%	n/a
Logistic Regression	90.0%	89.9%	n/a
XGBoost (tuned)	**94.0%**	**93.9%**	**0.982**

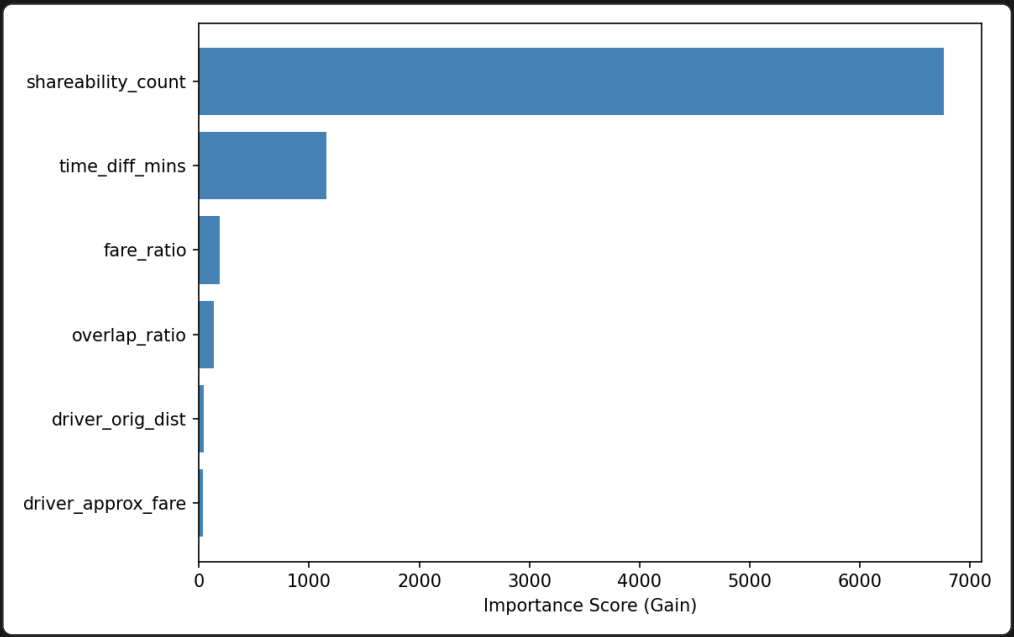
The rule-based model scores 93.3% but returns the same binary answer for every pair regardless of quality. XGBoost outputs a continuous probability (0 to 1) that the bipartite optimizer uses as an edge weight to find globally optimal assignments.

5-Fold CV: 85.2% (±17.9%) | MAE: 2.74 km | Best: depth=3, lr=0.05, n=50



ROC-AUC 0.982: 98.2% probability of ranking an accepted pair above a rejected pair. This is the key metric because bipartite matching uses scores as edge weights, not binary labels.

What the model actually learns to look at

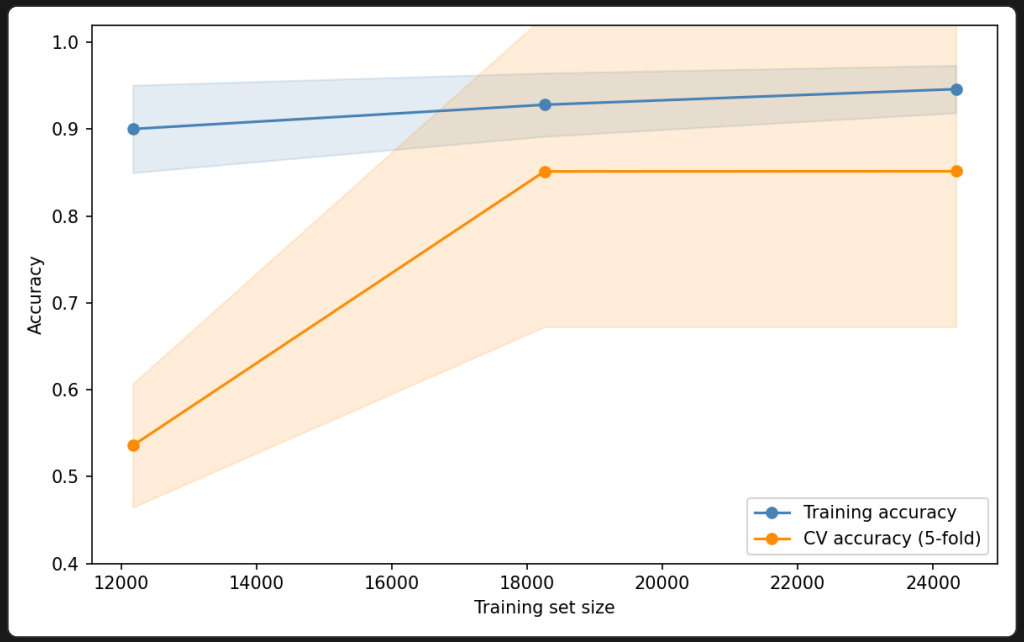


Feature importance by gain (XGBoost classifier). Higher = model splits on this feature more often and gains more information from it.

- **shareability_count [1]**: riders within 5 km in the same time batch. High value = driver is in a dense commuter corridor with many viable options.
- **time_diff_mins**: temporal compatibility. Pairs from different time windows also tend to have less spatial overlap, making this doubly informative.
- **fare_ratio** (rider_fare / driver_fare): direct economic incentive signal. Is the rider's fare worth the driver's detour?
- **overlap_ratio** = driver_orig_dist / (driver_orig_dist + detour), clamped [0,1]. ROC-AUC ablation shows +0.010 drop when removed, confirming route overlap is the primary geometric signal.

> Logistic regression cannot model that detour + fare interact non-linearly. A 2.5 km detour with a $30 fare should be accepted; the same detour with a $3 fare should not. XGBoost splits on both simultaneously.

Model validation: generalization and honest evaluation



Train and CV accuracy converge as dataset grows, confirming the model generalizes rather than memorizes

Driver-level train/test split

- Standard random split allows the same driver to appear in both train and test, inflating accuracy through entity leakage
- Our fix: split unique driver IDs 80/20 first, then assign all pairs for a given driver to one partition only
- Result: 30,425 train pairs, 7,677 test pairs. Zero driver ID overlap confirmed.

> The elevated CV standard deviation (±17.9%) reflects the driver-level splitting: fold sizes vary when driver trip counts are unequal. Expected to stabilize at full production scale (300K+ pairs).

Simulated system-level impact after bipartite matching

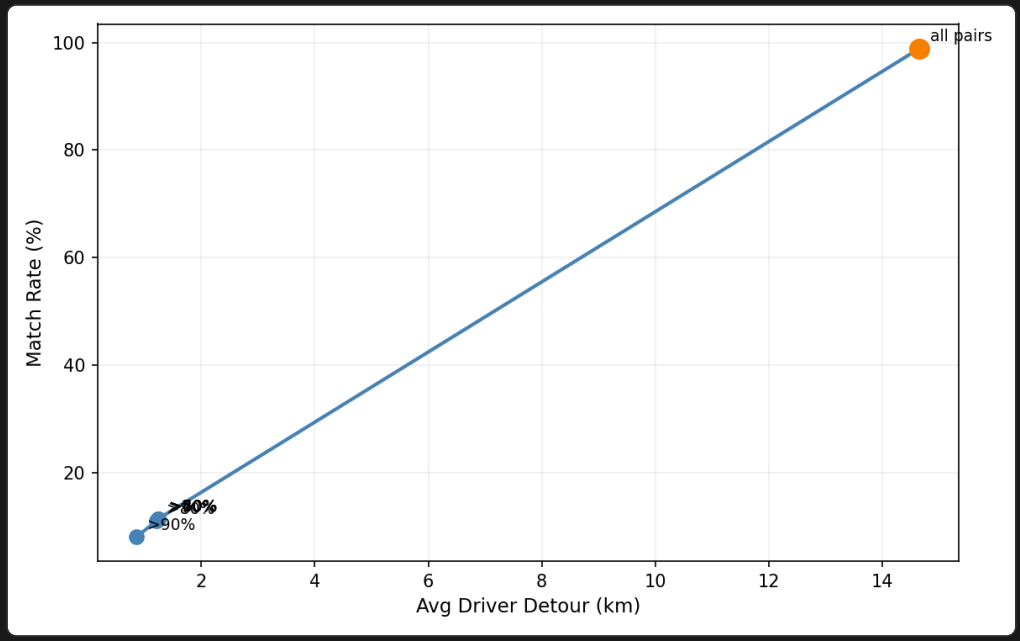
98.9%
Simulated match rate (3,969 test drivers)

+$5.75
Avg driver earnings per ride

-$5.75
Avg rider cost reduction

3.1 kg
CO_2 avoided per match (EPA: 404 g/mile)

Earnings assumption: driver receives 30% of rider's fare as the incentive split (avg rider fare $19.17 × 0.30 = $5.75). Actual split is a product decision.
CO_2 extrapolation: at 10% adoption in NYC (300K matched rides/day), estimated **930,000 kg CO_2 avoided daily** (rider's avoided solo taxi: fare/$2.50/mile × 404 g/mile, EPA 2023).



Operators can tune the confidence threshold. Higher threshold = fewer matches but shorter driver detour. Orange dot = no threshold (all feasible pairs matched).

Interpreting the results and ethical considerations

Key findings

- XGBoost (94.0% accuracy, ROC-AUC 0.982) outperforms rule-based (93.3%) and logistic regression (90.0%). The critical advantage is its continuous probability output: the rule-based model returns only 0/1 and cannot serve as bipartite edge weights.
- Adding the PM commute window (4–7 PM) alongside AM (7–10 AM) increased training pairs from ~1,290 to 38,102 (a 29x increase) while maintaining accuracy. Both rush hours are needed for a realistic product.
- ROC-AUC ablation: removing overlap_ratio causes the largest drop (+0.010), confirming that route overlap, not fare alone, is the primary driver of acceptance probability.

Ethical considerations

Privacy: Real GPS traces reveal home and work addresses. Strict encryption and data minimization are required before any deployment.

Fairness: Model trained on NYC data may underserve neighborhoods with fewer training trips.

Safety: P2P carpooling with strangers requires background checks and real-time ride tracking before launch.

Algorithmic bias: Higher fares make riders more attractive, which could disadvantage lower-income riders systematically.

Contributions and limitations

Three novel contributions

1. **Synthetic P2P training pipeline:** corridor-aware GPS synthesis from 3.5M public taxi records with polygon validation and spherical projection
2. **Dual-objective XGBoost:** regression (detour km) + classification (acceptance probability) fused into a single ranking score
3. **Learning-augmented bipartite matching:** ML scores as edge weights, training on all feasible pairs, provably optimal assignment at inference (Bjorck et al., AAAI 2021)

Honest limitations

Labels are simulated via behavioral model, not observed from real drivers

Geodesic (straight-line) distance used. Road-network routing via OSRM [4] would be more accurate, especially near rivers and bridges.

Trained and validated on NYC only. Feature abstraction (overlap_ratio is dimensionless) is designed to generalize, but cross-city empirical validation remains future work.

25% sample (38K pairs). Full dataset with 25K driver limit would yield ~300K pairs.

Future work: OSRM road routing [4] · Employer subsidies as ML features · Cross-city validation (Chicago) [9] · Real deployment feedback loop

github.com/PEM-Instruct/final-project-team-tle

References

Ride-Sharing and Matching

[1] Santi, P. et al. (2014). Quantifying the Benefits of Vehicle Pooling with Shareability Networks. *PNAS*, 111(37), 13290–13294.

[2] Bjorck, J. et al. (2021). Learning Augmented Methods for Matching. *AAAI*, pp. 15520–15521.

[3] Anagnostopoulos, T. & Ramson, S.R.J. (2025). A Vehicle Ride-Sharing Algorithm Assessing Passenger Satisfaction. *Future Transportation*, 5(2).

[4] Hsieh, F. (2017). Car Pooling Based on Trajectories of Drivers. *IEEE AINA*, pp. 972–979.

ML Methods and Tools

[5] Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD*, pp. 785–794.

[6] Grinsztajn, L. et al. (2022). Why Tree-Based Models Still Outperform Deep Learning on Tabular Data. *NeurIPS*, 35, 507–520.

[7] Omohundro, S.M. (1989). Five Balltree Construction Algorithms. *ICSI Tech Report* TR-89-063.

[8] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.

Data

[9] Santos, A. et al. (2011). Summary of Travel Trends: 2009 NHTS. *FHWA-PL-11-022*.